

HubIT MMS Interactive Workflows

A Developer & User Guide to the Maintenance Forms, Workflows, and MVC Patterns

Prepared For: Developers, System Admins, and Technicians

Date: June 2026

Target Module: MMS Forms & Task Schedulers

1. Functional File Mapping

The HubIT Maintenance Management System (MMS) relies on a structured set of forms and controllers located inside the forms_mms folder. Below is the mapping of active views, their specific roles, and their corresponding server-side endpoints:

File Name	Functional Description	Target Action / Route
addAssets.php	Fullpage form to register equipment profiles (ID, Serial, Location).	POST /form_mms/addAsset → AssetController::store
register_asset_modal.php	Modal popup to schedule maintenance, auto-calculating next dates.	POST /handler/registerAsset_handler.php
addMaintenance.php	Fullpage form to log or schedule specific maintenance actions.	POST /form_mms/addMaintenance → AssetMaintenanceController::store
checklist_template.php	Form to define step-by-step task checklists and intervals.	POST /form_mms/checklist_template → ChecklistTemplateController::store
create_checklist.php	Manager-facing view to initiate checklist logs for machines.	POST /form_mms/addMaintenance
checklist_lists.php	Grid listing active, pending, or completed checklists.	GET /form_mms/checklists → ChecklistController::index
maintenance_checklist.php	Tech-facing sheet to log results and pass/fail metrics.	POST /maintenance_checklist/update → MaintenanceChecklistController::update
routine_maintenance.php	Form to manually generate batch routine work orders.	POST /form_mms/routine_maintenance → RoutineMaintenanceController::generate

Note on Legacy/Unused Files:

- config_maint_form.php: Inactive legacy configuration template.
- routine_maintenance_preview.php: Misplaced controller code duplicate (now handled by app/controllers/RoutineMaintenanceController.php).

2. Overall Maintenance Workflow & Lifecycle

The HubIT MMS links equipment, schedule configurations, and physical checklist execution. The standard workflow flows through four distinct phases:

Phase 1: Inventory & Asset Setup

Before scheduling work, physical assets are registered using `addAssets.php`. Each machine is registered with a unique Asset ID (e.g. CNC-01) and location. Once saved, these populate the assets table scoped by the tenant's organization ID.

Phase 2: Defining Maintenance Templates

Using `checklist_template.php`, coordinators configure standard templates defining what must be done. They select a maintenance type (Preventive, Corrective, Inspection), set an interval in days (e.g., 30 days), and type out a series of step-by-step tasks (e.g., '1. Inspect oil level', '2. Verify emergency stop').

Phase 3: Scheduling & Dispatching Work

Work orders can be generated manually or automatically:

- **Manual Scheduling:** The user triggers `register_asset_modal.php`, chooses a machine and maintenance type, and the system automatically calculates the next maintenance date based on the template interval, inserting it as a 'scheduled' log.
- **Batch Generation:** Coordinators use `routine_maintenance.php` to select multiple machines, check constraints, and generate a batch of work orders.

Phase 4: Technician Checklist Execution

Technicians check off tasks in `maintenance_checklist.php`. They log remarks, input pass/fail/measurements, and click 'Mark as Completed' on their screen, which re-calculates the next maintenance cycle and archives the checklist results.

3. MVC Architecture & Request Flow

The system decouples data, processing, and interface layers using the Model-View-Controller pattern. Here is the request execution path using **Asset Registration** as a reference:

Step 1: Front Controller & Router

When the browser requests GET /mes/form_mms/addAsset, the webserver redirects the request to index.php. The routing engine loads app/routes.php and checks for matches. Upon finding the path matching GET /form_mms/addAsset, it instantiates AssetController and invokes create().

Step 2: Controller Response

The AssetController checks the user session, generates a CSRF token to prevent cross-site request forgery, and compiles local variables before loading the HTML form view:

```
include __DIR__ . '/../views/forms_mms/addAssets.php';
```

Step 3: Form Submission (POST)

The browser renders the form. The user types the information and clicks submit. A POST request goes to /mes/form_mms/addAsset, which is routed to AssetController::store(). The controller checks the CSRF token and calls the model:

```
// Inside app/controllers/AssetController.php
if ($this->model->addAsset($_POST)) {
    $_SESSION['success'] = 'Asset registered successfully';
    header('Location: /mes/form_mms/addAsset');
    exit;
}
```

Step 4: Model Execution & Database Write

The AssetModel.php interacts with PostgreSQL. It prepares a parameterized SQL query using PDO to write the record cleanly while preventing SQL injection:

```
// Inside app/models/AssetModel.php
public function addAsset($data) {
    $stmt = $this->db->prepare("INSERT INTO assets (asset_id, asset_name, org_id) VALUES (?, ?, ?)");
    return $stmt->execute([$data['asset_id'], $data['asset_name'], $data['tenant_id']]);
}
```